PES University - Department of Computer Science and Engineering

DBMS Experiential Learning – Level 3 Project Report (Soft Copy)

ReliefChain: Disaster Relief Resource Allocation & Routing System

Team Size: 2 Members
Database Paradigms: Dual-Paradigm: Relational SQL (PostgreSQL) + Vector Database (pgvector) + Document NoSQL (MongoDB)
Application Stack: Next.js 15 (React 19, TypeScript), Node.js, Docker Compose
Academic Year: 2026

1. Executive Summary & Problem Definition

Natural catastrophes (cyclones, catastrophic floods, landslides) devastate civil infrastructure, disrupting supply networks, communication lines, and shelter access. In disaster management, relief organizations face severe operational bottlenecks:

1. **Supply & Demand Mismatches**: Supplies are distributed arbitrarily without real-time tracking of camp capacities or specific supply deficits.
2. **Race Conditions & Stock Double-Allocation**: Multiple field coordinators dispatch supplies from the same depot simultaneously, causing inventory deficits.
3. **Unstructured Distress Processing**: Victims and field volunteers broadcast panic messages through SMS, voice-to-text, and messaging apps containing unstructured, fuzzy requirements that cannot be queried through standard relational SQL predicates.
4. **Logistics Routing Inefficiencies**: Dispatches are organized without spatial optimization, ignoring great-circle geographic proximity and fleet vehicle payload capacities.

ReliefChain is an enterprise disaster response platform engineered to resolve these challenges through a dual-paradigm database architecture combining **PostgreSQL (with ACID transactions and pgvector embeddings)** and **MongoDB (for unstructured high-velocity drone telemetry and event logs)**.

2. Team Formation & Individual Contributions

Per the course guideline requiring clearly defined responsibilities for the 2-member team:

Team Member	Primary Specialization	Specific Database Contributions	Application & API Contributions
Member 1	**Database Architect & Backend Lead**	- Conceptual ER modeling and 3NF Relational Schema design. - DDL implementation with primary keys, composite keys, referential integrity (`ON DELETE RESTRICT/CASCADE`), and CHECK constraints. - ACID Stored Procedure `sp_process_resource_dispatch` with row-level locks (`SELECT ... FOR UPDATE`). - Geospatial Haversine Stored Function `fn_find_nearest_warehouse_with_stock` . - SQL Triggers for negative inventory prevention and audit trail logging. - Multi-table Views (`v_camp_deficit_matrix` , `v_fleet_readiness` , `v_active_disaster_dashboard`).	- Database connection pool orchestration (`pg.Pool`). - Backend API routes for inventory balancing (`/api/inventory`), relief requests (`/api/requests`), and ACID dispatch executions (`/api/dispatch`). - SQL query benchmark suite & performance indexing.
Member 2	**Full-Stack & NoSQL/Vector Specialist**	- Vector Database Architecture using **pgvector** (`vector(384)` dimension column). - HNSW index creation (`idx_sos_embedding_hnsw`) with cosine similarity distance operator (`<=>`). - MongoDB NoSQL schema setup for drone reconnaissance and sensor telemetry. - MongoDB Multi-Stage Aggregation Pipeline (`$match` → `$group` → `$project` → `$sort`). - Stored Function `fn_calculate_camp_urgency_score` .	- Next.js 15 War Room dashboard & interactive UI components. - Semantic vector ingestion engine & Cosine Similarity ranking (`/api/semantic-search` , `/api/sos`). - JWT Authentication & Role-Based Access Control (RBAC) middleware (`/api/auth/login`). - Interactive Evaluation SQL Lab workbench (`/api/sql-lab`). - Docker Compose orchestration & automated seed suite.

3. Database Choice & Justification

The project satisfies both **Option 1 (SQL + NoSQL)** and **Option 2 (SQL + Vector Database)** by uniting PostgreSQL (Relational + pgvector) with MongoDB:

Paradigm	Selected Engine	Purpose in ReliefChain	Technical Justification
Relational SQL	**PostgreSQL 16**	Core transactional entity management (camps, warehouses, dispatches, inventory, fleet).	Critical for **ACID compliance**, strict referential integrity, row-level locking (`FOR UPDATE`) to prevent inventory over-allocation, and complex relational joins across entities.
Vector Database	**PostgreSQL pgvector**	Semantic triage and natural language matching of unstructured victim SOS messages.	Standard relational queries (`LIKE '%water%'`) miss context like *"dehydration in elderly shelter"*. pgvector maps natural text to 384-dimensional dense vectors and performs fast Approximate Nearest Neighbor (ANN) search via HNSW indexing.
NoSQL Document	**MongoDB 7.0**	IoT Drone aerial surveys, thermal sensor streams, road blockage reports, and rapid situational event feeds.	Drone reconnaissance telemetry generates polymorphic JSON payloads with variable sensor arrays (thermal cameras, lidar, water level meters). MongoDB's schema-less document architecture and aggregation pipeline process this high-velocity time-series stream effortlessly.

4. Entity-Relationship (ER) Diagram & Relational Schema

4.1 Mermaid ER Diagram

```
erDiagram
    USERS |├──o{ RELIEF_CAMPS : "assigned/monitors"
    USERS |├──o{ WAREHOUSES : "manages"
    USERS |├──o{ DISPATCH_SHIPMENTS : "authorizes"

    DISASTER_ZONES |├──o{ RELIEF_CAMPS : "contains"
    DISASTER_ZONES |├──o{ EMERGENCY_SOS_SIGNALS : "originates_in"

    RELIEF_CAMPS |├──o{ RELIEF_REQUESTS : "initiates"

    RESOURCE_CATEGORIES |├──o{ WAREHOUSE_INVENTORY : "stocked_as"
    RESOURCE_CATEGORIES |├──o{ RELIEF_REQUESTS : "demanded_as"
    RESOURCE_CATEGORIES |├──o{ DISPATCH_ITEMS : "manifested_in"
    RESOURCE_CATEGORIES |├──o{ EMERGENCY_SOS_SIGNALS : "matched_to"

    WAREHOUSES |├──o{ WAREHOUSE_INVENTORY : "stores"
    WAREHOUSES |├──o{ VEHICLE_FLEET : "depots"
    WAREHOUSES |├──o{ DISPATCH_SHIPMENTS : "originates"

    VEHICLE_FLEET |├──o{ DISPATCH_SHIPMENTS : "transports"

    RELIEF_REQUESTS |├──o{ DISPATCH_SHIPMENTS : "fulfilled_by"

    DISPATCH_SHIPMENTS |├──┤{ DISPATCH_ITEMS : "contains"

    USERS {
        int id PK
        string email UQ
        string password_hash
        string full_name
        string role
        string phone
        int assigned_zone_id FK
    }

    DISASTER_ZONES {
        int id PK
        string name
        string disaster_type
        string severity_level
        decimal latitude
        decimal longitude
        decimal radius_km
        string status
    }

    RELIEF_CAMPS {
        int id PK
        int zone_id FK
        string name
        string camp_type
        string contact_person
        string contact_phone
        int capacity
        int current_occupancy
        string status
    }

    WAREHOUSES {
```

```
        int id PK
        string name
        string location_name
        decimal latitude
        decimal longitude
        int capacity_sqft
        int manager_id FK
        boolean is_operational
    }

    RESOURCE_CATEGORIES {
        int id PK
        string code UQ
        string name
        string category_group
        string unit_of_measure
        boolean is_perishable
        decimal weight_kg_per_unit
        int priority_rank
    }

    WAREHOUSE_INVENTORY {
        int id PK
        int warehouse_id FK
        int category_id FK
        int available_quantity
        int reserved_quantity
        int minimum_threshold
    }

    RELIEF_REQUESTS {
        int id PK
        int camp_id FK
        int category_id FK
        int requested_quantity
        int approved_quantity
        string urgency_level
        string status
    }

    EMERGENCY_SOS_SIGNALS {
        int id PK
        int zone_id FK
        string sender_name
        string sender_phone
        decimal latitude
        decimal longitude
        int victim_count
        text raw_message
        vector embedding
        string detected_urgency
        int matched_category_id FK
        string status
    }

    VEHICLE_FLEET {
        int id PK
        string vehicle_number UQ
        string vehicle_type
        decimal max_payload_kg
        int home_warehouse_id FK
        string status
    }

    DISPATCH_SHIPMENTS {
```

```
        int id PK
        string tracking_code UQ
        int request_id FK
        int warehouse_id FK
        int vehicle_id FK
        int dispatched_by FK
        string delivery_status
        decimal route_distance_km
    }

    DISPATCH_ITEMS {
        int id PK
        int shipment_id FK
        int category_id FK
        int quantity_loaded
    }
```

4.2 Relational Schema (3NF Formal Representation)

1. **`users`** (`id` [PK], `email` [UQ], `password_hash`, `full_name`, `role`, `phone`, `assigned_zone_id` [FK → `disaster_zones.id`], `is_active`, `created_at`, `updated_at`)
2. **`disaster_zones`** (`id` [PK], `name`, `disaster_type`, `severity_level`, `latitude`, `longitude`, `radius_km`, `estimated_affected_pop`, `status`, `declared_at`)
3. **`relief_camps`** (`id` [PK], `zone_id` [FK → `disaster_zones.id`], `name`, `camp_type`, `contact_person`, `contact_phone`, `latitude`, `longitude`, `capacity`, `current_occupancy`, `status`, `requires_cold_storage`, `created_at`)
4. **`warehouses`** (`id` [PK], `name`, `location_name`, `latitude`, `longitude`, `capacity_sqft`, `manager_id` [FK → `users.id`], `is_operational`, `created_at`)
5. **`resource_categories`** (`id` [PK], `code` [UQ], `name`, `category_group`, `unit_of_measure`, `is_perishable`, `weight_kg_per_unit`, `priority_rank`)
6. **`warehouse_inventory`** (`id` [PK], `warehouse_id` [FK → `warehouses.id`], `category_id` [FK → `resource_categories.id`], `available_quantity`, `reserved_quantity`, `minimum_threshold`, `last_restocked_at`) — *Unique Composite Key: (`warehouse_id`, `category_id`)*
7. **`relief_requests`** (`id` [PK], `camp_id` [FK → `relief_camps.id`], `category_id` [FK → `resource_categories.id`], `requested_quantity`, `approved_quantity`, `urgency_level`, `status`, `notes`, `created_at`, `fulfilled_at`)
8. **`emergency_sos_signals`** (`id` [PK], `zone_id` [FK → `disaster_zones.id`], `sender_name`, `sender_phone`, `latitude`, `longitude`, `victim_count`, `raw_message`, `embedding` [vector(384)], `detected_urgency`, `matched_category_id` [FK → `resource_categories.id`], `status`, `created_at`)
9. **`vehicle_fleet`** (`id` [PK], `vehicle_number` [UQ], `vehicle_type`, `max_payload_kg`, `current_latitude`, `current_longitude`, `home_warehouse_id` [FK → `warehouses.id`], `status`, `last_ping_time`)
10. **`dispatch_shipments`** (`id` [PK], `tracking_code` [UQ], `request_id` [FK → `relief_requests.id`], `warehouse_id` [FK → `warehouses.id`], `vehicle_id` [FK → `vehicle_fleet.id`], `dispatched_by` [FK → `users.id`], `departure_time`, `estimated_arrival`, `actual_arrival`, `delivery_status`, `route_distance_km`, `carbon_footprint_kg`, `delivery_notes`)
11. **`dispatch_items`** (`id` [PK], `shipment_id` [FK → `dispatch_shipments.id`], `category_id` [FK → `resource_categories.id`], `quantity_loaded`) — *Unique Composite Key: (`shipment_id`, `category_id`)*
12. **`allocation_audit_log`** (`id` [PK], `entity_name`, `entity_id`, `action_type`, `performed_by` [FK → `users.id`], `old_state` [JSONB], `new_state` [JSONB], `timestamp`)

---

5. Data Dictionary & Integrity Constraints

Table Name	Column Name	Data Type	Nullable	Key / Constraint	Functional Description
`disaster_zones`	`disaster_type`	VARCHAR(50)	NO	CHECK (`FLOOD` , `EARTHQUAKE` , `CYCLONE` , etc.)	Catastrophic incident classification
`disaster_zones`	`severity_level`	VARCHAR(20)	NO	CHECK (`CRITICAL` , `HIGH` , `MODERATE` , `LOW`)	Government-declared emergency rating
`disaster_zones`	`latitude`	DECIMAL(10, 7)	NO	CHECK (`-90.0` to `90.0`)	Geographic coordinates
`relief_camps`	`current_occupancy`	INT	NO	CHECK (`≥ 0`)	Number of sheltered evacuees
`relief_camps`	`chk_occupancy_capacity`	TABLE CONSTRAINT	—	CHECK (`current_occupancy ≤ capacity * 1.5`)	Restricts overflow beyond 150% capacity
`resource_categories`	`priority_rank`	INT	NO	CHECK (`1` to `5`)	1 = Life Critical (Blood/Water), 5 = Secondary
`warehouse_inventory`	`available_quantity`	INT	NO	CHECK (`≥ 0`)	Unallocated physical stock in depot
`warehouse_inventory`	`uq_wh_cat`	TABLE CONSTRAINT	—	UNIQUE (`warehouse_id` , `category_id`)	Prevents duplicate stock rows
`emergency_sos_signals`	`embedding`	vector(384)	YES	HNSW Cosine Index	Dense semantic vector for NLP similarity
`vehicle_fleet`	`vehicle_type`	VARCHAR(40)	NO	CHECK (`HEAVY_TRUCK` , `DRONE_FLEET` , etc.)	Transport modality
`dispatch_shipments`	`tracking_code`	VARCHAR(40)	NO	UNIQUE	Alphanumeric mission identifier

6. Advanced SQL & Database Features Implementation

6.1 ACID Transaction Stored Procedure (`sp_process_resource_dispatch`)

The allocation of disaster resources must guarantee atomicity to prevent double-spending stock. The procedure implements:

1. `SELECT ... FOR UPDATE` locks on both `relief_requests` and `warehouse_inventory` rows.
2. Verification of vehicle payload vs weight ($Quantity \times Unit Weight \le Payload$).
3. Stock deduction from `available_quantity` to `reserved_quantity`.
4. Generation of a unique dispatch shipment and items manifest.
5. Fleet vehicle state transition from `AVAILABLE` to `IN_TRANSIT`.
6. Automatic creation of audit log record in `allocation_audit_log`.

6.2 Stored Functions

- `fn_find_nearest_warehouse_with_stock(p_camp_id, p_category_id, p_required_qty)`: Implements the **Haversine Great-Circle Distance Formula** in PL/pgSQL to calculate spherical distances between the camp and operational warehouses, returning depots sorted ascending by distance.
- `fn_calculate_camp_urgency_score(p_camp_id)`: Dynamically computes an urgency score from 0 to 100 based on occupancy percentage (40% weight), zone severity (35% weight), and pending critical requests (25% weight).

6.3 Database Triggers

- `trg_chk_inventory_levels`: Before insert/update trigger on `warehouse_inventory` rejecting any negative balance at the engine level.
- `trg_audit_request`: After update trigger on `relief_requests` logging all status transitions into `allocation_audit_log`.
- `trg_shipment_delivery`: Before update trigger on `dispatch_shipments` that, upon marking a shipment `DELIVERED`, automatically frees the vehicle back to `AVAILABLE` and marks the parent request `FULFILLED`.

6.4 Complex Analytical Views

- `v_camp_deficit_matrix`: Computes outstanding itemized camp deficits ($Deficit = Requested - Delivered$).
- `v_active_disaster_dashboard`: Aggregates occupancy percentages, critical request counts, and pending SOS per catastrophe zone.
- `v_fleet_readiness`: Real-time vehicle deployment, active mission codes, and payload utilization.

---

7. Vector Database & Semantic AI Search (pgvector)

1. **Input Data**: Unstructured, conversational, or panicked emergency distress messages (e.g. "*Our roof is flooded, my diabetic mother needs her insulin injections immediately*").
2. **Embedding Model**: 384-dimensional dense projection compatible with `sentence-transformers/all-MiniLM-L6-v2`.
3. **Storage**: PostgreSQL `vector(384)` column in `emergency_sos_signals`.
4. **Index Architecture**: **HNSW (Hierarchical Navigable Small World)** with `m = 16, ef_construction = 64`.
5. **Similarity Metric**: Cosine Distance operator (`<=>`):
   $$\text{Cosine Similarity} = 1 - (u \cdot v)$$
6. **Execution Query**:

```
SELECT id, sender_name, raw_message, 1 - (embedding <=> $1::vector) AS similarity_score
FROM emergency_sos_signals
WHERE embedding IS NOT NULL
ORDER BY embedding <=> $1::vector ASC
LIMIT 5;
```

---

8. NoSQL Database & Multi-Stage Aggregation Pipeline (MongoDB)

IoT Drone reconnaissance surveys and real-time sensor streams are ingested into MongoDB collections:

- `drone_recon_telemetry`: Stores battery metrics, 4K camera damage classification, detected thermal human survivors, and impassable road blockages with water depths.
- `incident_event_feed`: High-velocity unstructured event stream from weather radar, power grid SCADA, and police radio.

Multi-Stage Aggregation Pipeline:

```
[
  // Stage 1: Filter missions from the active 24-hour crisis window
  { $match: { timestamp: { $gte: new Date(Date.now() - 24 * 60 * 60 * 1000) } } },
  // Stage 2: Group by disaster zone and compute aggregates
  {
    $group: {
      _id: '$zone_id',
      zone_name: { $first: '$zone_name' },
      total_missions: { $sum: 1 },
      avg_hazard_score: { $avg: '$survey_results.hazard_score' },
      max_hazard_score: { $max: '$survey_results.hazard_score' },
      total_submerged_structures: { $sum: '$survey_results.submerged_structures_count' },
      all_blockages: { $push: '$survey_results.road_blockages' },
      survivor_detections: { $push: '$survey_results.thermal_survivor_detections' }
    }
  },
  // Stage 3: Reshape document and calculate nested arrays
  {
    $project: {
      zone_id: '$_id',
      zone_name: 1,
      total_missions: 1,
      avg_hazard_score: { $round: ['$avg_hazard_score', 2] },
      total_submerged_structures: 1,
      blockage_count: { $size: { $reduce: { input: '$all_blockages', initialValue: [], in: { $concatArrays: [
      total_thermal_survivors: { $sum: '$survivor_detections.count' }
    }
  },
  // Stage 4: Order by hazard score descending
  { $sort: { avg_hazard_score: -1 } }
]
```

9. Test Cases & Verification Results

Test Case ID	Feature Tested	Input / Action	Expected Result	Actual Result	Status
TC-01	Negative Inventory Constraint	`UPDATE warehouse_inventory SET available_quantity = -50 WHERE id = 1;`	DB trigger rejects update and raises exception	`Database Integrity Violation: available_quantity cannot be negative`	**PASS**
TC-02	ACID Dispatch Stored Procedure	`CALL sp_process_resource_dispatch(3, 1, 1, 30, 1, NULL);`	Stock decreases from 120 to 90, tracking code generated, audit log entry created	Stock updated, code `RC-20261031-7853` assigned, audit log populated	**PASS**
TC-03	Haversine Distance Function	`SELECT * FROM fn_find_nearest_warehouse_with_stock(1, 1, 100);`	Returns warehouses ordered ascending by spherical km distance	Depot 1 (8.95 km), Depot 2 (285.07 km), Depot 3 (426.44 km)	**PASS**
TC-04	Delivery Trigger Execution	Update shipment status to `DELIVERED`	Vehicle status set to `AVAILABLE`, request status set to `FULFILLED`	Trigger executed immediately, vehicle freed	**PASS**
TC-05	pgvector Cosine Search	Query: *"elderly diabetic insulin"*	Returns distress calls with insulin/medical keywords ranked by similarity	SOS #3 and #2 matched with > 78% similarity score	**PASS**
TC-06	MongoDB Aggregation Pipeline	Execute 4-stage pipeline on `drone_recon_telemetry`	Aggregates hazard score, submerged buildings, and survivor counts by zone	Successfully returned grouped JSON metrics ordered by hazard score	**PASS**
TC-07	Role-Based Access Control	Login as `FIRST_RESPONDER`	Restricts access to sensitive global fleet configuration while allowing field triage	Token contains role `FIRST_RESPONDER` and enforces API permissions	**PASS**

10. Conclusion

ReliefChain demonstrates the application of multi-paradigm database systems to solve a high-stakes, real-world disaster logistics challenge. By utilizing **PostgreSQL** for relational integrity and ACID-safe resource dispatch, **pgvector** for semantic

distress triage, and **MongoDB** for unstructured drone sensor telemetry, the project goes far beyond a basic CRUD application and fulfills all criteria for the PES University Level 3 DBMS Experiential Learning curriculum.